Black-Box Reverse Engineering of CPU Microarchitecture

Reorder Buffer Capacity, Physical Register File Size, and Translation Lookaside Buffer Hierarchy

Native Linux versus Intel SGX (Gramine) Execution on an Intel Xeon Gold 5420+

Ian Ongwacho
ECE 492 — Microarchitecture Reverse Engineering
North Carolina State University

Homework 3 — Track B (Undergraduate Version) • November 3, 2026

Abstract

I examined three black-box microarchitecture questions on the course server `artemisia` (Intel Xeon Gold 5420+, Sapphire Rapids): reorder buffer (ROB) capacity, physical register file (PRF) behavior for integer and vector registers, and the data translation lookaside buffer (DTLB) hierarchy. This is Homework 3, **Track B (undergraduate)**, covering Questions 4, 5, and 6 for 30, 35, and 35 points, respectively. Under the instructor's undergraduate modification, one course machine suffices for each selected experiment. I used serialized timestamp-counter measurements, randomized pointer chasing, control workloads, repeated trials, and selected native-versus-Intel-SGX comparisons using Gramine.

For the ROB, a candidate latency discontinuity occurred between 480 and 496 inserted NOPs in SGX; the external 512-entry Golden Cove reference remains a qualified prediction rather than an independently established count. For PRF pressure, native timing transitions occurred near 212 integer-add and 244 XMM-add filler instructions, with broadly corresponding but noisier SGX behavior. These are instruction thresholds, not direct physical-register counts, and integer arithmetic also modifies flags. For 4-KiB translations, native and enclave data both showed a sharp first transition near 96 distinct pages and larger-working-set behavior consistent with, but not sufficient to prove, a second-level translation resource around 2,048 entries. SGX incurred substantially higher latency in the large-page-count regime. Throughout, I distinguish observed thresholds, external architectural references, and unresolved experimental confounders.

Table 1. Results at a glance. Thresholds for ROB and PRF count inserted instructions; TLB thresholds count distinct 4-KiB pages.

Experiment	Primary observation	Supported claim
Q4: ROB (30 pts)	SGX latency jump at 480–496 inserted NOPs; weaker native change	Consistent with a 512-entry reference; exact capacity not isolated
Q5: PRF (35 pts)	Native knees near 212 (GPR) and 244 (XMM) filler instructions; similar SGX neighborhoods	Class-dependent effective limits; exact PRF sizes unresolved
Q6: TLB (35 pts)	Shared first transition near 96 pages; broad rise across 1,536–3,072 pages; SGX much slower at large working sets	L1 DTLB ≈96 entries (stronger evidence); STLB near 2,048 plausible (weaker)

Contents

1 Scope, Platform, and Conventions **5**
1.1 Selected experiments . . . 5
1.2 Experimental platform . . . 5
1.3 Evidence labels used in this report . . . 5

2 Common Measurement Infrastructure **6**
2.1 Serialized timing . . . 6
2.2 Randomized pointer chains . . . 6
2.3 The two-chain window probe . . . 7
2.4 Controls, trials, and statistics . . . 7

3 Question 4: Reorder Buffer (ROB) Capacity **8**
3.1 Probe implementation . . . 8
3.2 Run configurations . . . 9
3.3 Results . . . 9
3.4 Native versus SGX . . . 13
3.4.1 Enclave configuration and protocol . . . 13
3.4.2 Measured enclave timing . . . 14
3.4.3 Direct comparison . . . 16
3.4.4 Assembly verification of the transition region . . . 16
3.5 Challenges and limitations . . . 17
3.6 Capacity estimate and prediction . . . 17

4 Question 5: Physical Register File (PRF) Characterization **18**
4.1 Hypothesis and design . . . 18
4.2 Native observations . . . 19
4.3 Enclave results and comparison . . . 21

5 Question 6: Data TLB Hierarchy and Capacity **22**
5.1 Translation-sensitive methodology . . . 22
5.2 Memory setup and huge-page control . . . 23
5.3 Sweeps and platforms . . . 23
5.4 First-level DTLB transition . . . 24
5.5 Second-level translation pressure . . . 25
5.6 Full sweep and the page-walk regime . . . 26

6 Cross-Experiment Interpretation **28**

7 Conclusion **29**

A Additional Code: Measurement Drivers **29**

B Reproducibility and Artifact Inventory **30**

C AI-Use Disclosure (Track B) **31**

1 Scope, Platform, and Conventions

1.1 Selected experiments

For Homework 3, I selected Track B and the three microarchitecture reverse-engineering experiments in Table 2. Under the instructor's announced undergraduate simplification, each selected experiment is performed on only one course machine rather than two. Where supported, experiments are compared between native Linux execution and Intel Software Guard Extensions (SGX) enclave execution.

Table 2. Selected Track B experiments and point allocation.

Question	Experimental objective	Points
4	Reverse engineer the reorder buffer (ROB) size.	30
5	Reverse engineer the physical register file (PRF) size and identify the register class.	35
6	Reverse engineer the number of translation lookaside buffer (TLB) levels and determine the size of each level.	35
	Total selected points	**100**

1.2 Experimental platform

All experiments ran on the course server `artemisia`. The `lscpu` output identified an Intel Xeon Gold 5420+ with two sockets and 112 logical CPUs (56 physical cores in total, two hardware threads per core). Intel lists the 5420+ as a 4th-generation Xeon Scalable processor of the Sapphire Rapids family [3].

Table 3. Platform and common experimental configuration.

Item	Value
Host	`artemisia` (course server)
Processor	Intel Xeon Gold 5420+ (Sapphire Rapids; Golden Cove cores)
Topology	2 sockets, 112 logical CPUs (56 cores × 2 threads)
CPU affinity	Logical CPU 10 (`taskset -c 10`) for every run
Compiler flags	`gcc -O2 -Wall -Wextra -std=gnu11 -fno-unroll-loops`
Timer	`LFENCE`-serialized `RDTSC`/`RDTSCP` (TSC ticks)
Enclave runtime	Gramine library OS on hardware SGX (user-local install)
Instrumentation	Hardware performance counters (`perf`) prohibited

1.3 Evidence labels used in this report

Because black-box timing yields *thresholds* rather than direct capacity readouts, every quantitative statement is tagged by its epistemic status.

Observed A value measured directly in my runs (a latency, a threshold location, a control result).

Reference A published architectural figure (for example, a 512-entry Golden Cove ROB [4, 5]) used for comparison only; it is *not* verified by these measurements.

Inferred An interpretation connecting an observation to a hardware resource. These are explicitly qualified and are not treated as established capacities.

2 Common Measurement Infrastructure

All three experiments share the same measurement approach: serialized timing, randomized dependent-load (pointer-chasing) chains, repeated trials summarized by robust statistics, and a control workload.

2.1 Serialized timing

A reorder buffer tracks in-flight operations and ensures that architectural retirement respects program order, even when execution is out of order. A long-latency load may obstruct retirement of younger operations while allowing independent younger instructions to enter the machine. If a later memory operation lies beyond the available window, its issue can be delayed, reducing the overlap between otherwise independent accesses [6].

To observe this I time each loop with `LFENCE`-bracketed `RDTSC/RDTSCP` (Listing 1). The values are *TSC ticks*, not guaranteed core clock cycles under dynamic frequency scaling.

```c
static inline uint64_t x86_tsc_start(void) {
    _mm_lfence();                // drain older instructions
    uint64_t t = __rdtsc();
    _mm_lfence();                // keep younger work behind the read
    return t;
}

static inline uint64_t x86_tsc_stop(void) {
    unsigned aux;
    uint64_t t = __rdtscp(&aux);  // waits for prior instructions
    _mm_lfence();
    return t;
}
```

Listing 1. Serialized timestamp primitives (shared by `ROB.c`, `PRF.c`, and `TLB_improved.c`).

2.2 Randomized pointer chains

Each node holds a pointer to its successor, so every load's address depends on the previous load's value. A 32-bit xorshift generator and a Fisher–Yates shuffle produce a single randomized circular chain that defeats stride prefetching (Listing 2). In the ROB and PRF experiments each node occupies one 64-byte cache line and each chain contains 2^{20} nodes (≈ 64 MiB per chain; ≈ 128 MiB for the pair). This layout reduces spatial locality, but it does not guarantee a DRAM miss on every load.

```c
typedef struct node {
    struct node *next;
```

```c
    unsigned char padding[64 - sizeof(void *)];   // one cache line per node
} Struct1;
_Static_assert(sizeof(Struct1) == 64, "Node size must be 64 bytes");

static uint32_t xorshift32(uint32_t *state) {
    uint32_t x = *state;
    x ^= x << 13;  x ^= x >> 17;  x ^= x << 5;
    return *state = x;
}

// Fisher-Yates shuffle of order[0..n-1] ...
for (size_t i = n - 1; i > 0; i--) {
    size_t j = (size_t)(xorshift32(&state) % (uint32_t)(i + 1));
    size_t tmp = order[i]; order[i] = order[j]; order[j] = tmp;
}
// ... then link the shuffled nodes into one cycle
for (size_t i = 0; i < n; i++)
    nodes[order[i]].next = &nodes[order[(i + 1) % n]];
```

Listing 2. Cache-line-sized node and randomized cycle construction (excerpt from `ROB.c`; identical in `PRF.c`).

2.3 The two-chain window probe

The ROB and PRF experiments use the same probe (Fig. 1). Each iteration loads from chain A (long latency), executes N independent filler instructions, then loads from an independent chain B. If the filler fits inside the out-of-order window, load B issues while load A is still outstanding and the two latencies overlap. Once the fillers exhaust a limiting resource, allocation stalls, load B waits for load A to retire, and iteration time increases. Only the *type* of filler changes between experiments: NOPs consume ROB entries only, whereas register-writing instructions additionally consume physical registers of a specific class.

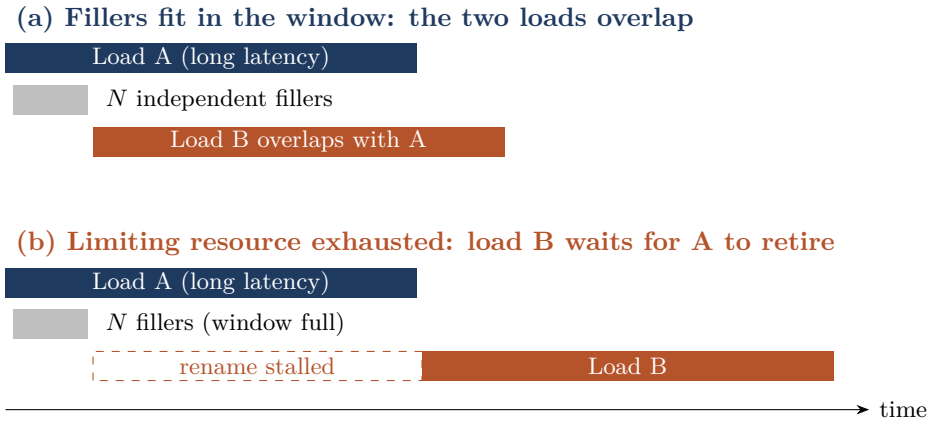


Figure 1. Principle of the two-chain probe. Panel (b) shows the iteration-time increase that signals a limiting resource.

2.4 Controls, trials, and statistics

Each probe has a matching *control* that executes the same filler sequence without the pointer loads. The control estimates the cost of the filler sequence in isolation. It is informative but is *not* an additive overhead that can be subtracted from the two-chain time, because filler execution and memory stalls overlap.

Results are summarized with the per-point *median* and the interquartile range (IQR; 25th–75th percentile). The median limits the leverage of occasional slow trials, and shaded envelopes in the figures are IQRs, not confidence intervals. To reduce ordering bias, sweep direction and the order of test and control alternate between trials. Hardware performance counters were prohibited, so these data cannot directly measure retirement stalls, cache misses, or ROB occupancy.

3 Question 4: Reorder Buffer (ROB) Capacity

3.1 Probe implementation

For each filler count N, a separately compiled function is generated with a macro and the assembler's `.rept` directive (Listing 3). The loop body is a dependent load from chain A, N NOPs, and a load from chain B. The NOP-only control (Listing 4) executes the same NOP count without loads. The benchmark returns elapsed ticks divided by loop iterations, indexed by NOP count and trial.

```
#define DEFINE_ROB_TEST(N)                                          \
static __attribute__((noinline))                                    \
double rob_test_##N(Struct1 *a, Struct1 *b, uint64_t steps) {       \
    uint64_t t0 = x86_tsc_start();                                  \
    for (uint64_t i = 0; i < steps; i++) {                          \
        __asm__ volatile (                                          \
            "mov (%0), %0\n\t"            /* load A (chain A) */\
            ".rept " STRINGIFY(N) "\n\t"                            \
            "nop\n\t"                     /* N fillers        */\
            ".endr\n\t"                                             \
            "mov (%1), %1\n\t"            /* load B (chain B) */\
            : "+r"(a), "+r"(b) : : "memory");                       \
    }                                                               \
    uint64_t t1 = x86_tsc_stop();                                   \
    sink_a = a;  sink_b = b;           /* keep chains observable */\
    return (double)(t1 - t0) / steps;                               \
}
```

Listing 3. Core of the two-chain ROB probe (`ROB.c`).

```
#define DEFINE_NOP_CONTROL(N)                                       \
static __attribute__((noinline))                                    \
double nop_control_##N(uint64_t steps) {                            \
    uint64_t t0 = x86_tsc_start();                                  \
    for (uint64_t i = 0; i < steps; i++) {                          \
        __asm__ volatile (".rept " STRINGIFY(N) "\n\t"              \
                          "nop\n\t" ".endr\n\t" : : : "memory");\
    }                                                               \
    uint64_t t1 = x86_tsc_stop();                                   \
    return (double)(t1 - t0) / steps;                               \
}

// One list generates both test and control tables, so they cannot diverge.
#define TEST_COUNTS(X) \
    X(0) X(16) X(32) X(64) X(96) X(112) X(128) X(144) X(160) X(192) X(224) \
    X(256) X(320) X(384) X(448) X(480) X(496) X(512) X(528) X(544) X(576)  \
    X(640) X(768)
```

Listing 4. NOP-only control and the NOP sweep list (`ROB.c`).

3.2 Run configurations

Two runs were performed (Table 4). Run 1 used the original compact, pointer-only node layout. Run 2 adopted the 64-byte node layout, a longer timed loop, more trials, a wider sweep with extra sampling around 512, and an NOP-only control.

Table 4. ROB run configurations.

Parameter	Run 1	Run 2
Node layout	Compact (pointer only)	64-byte nodes, 2^{20} per chain, 64-byte aligned
Measured iterations	10,000	100,000 (plus 10,000 warm-up)
Trials per point	7	15
NOP sweep	0–512	0–768, denser near 512
Sweep order	Ascending	Alternating each trial
NOP-only control	—	Yes

3.3 Results

Run 1 (naive layout). Figure 2 shows that Run 1 was approximately flat at 16–17 ticks per iteration for 0–96 NOPs, then increased with NOP count to about 71 ticks at 512 NOPs. The early rise at 96–128 NOPs is tempting to read as an instruction-window limit, but it equally arises because the filler sequence's front-end and execution costs become visible when the load latency is short. The first zero-NOP measurement and several points near 224 NOPs were abnormally high, which motivated medians over repeated trials.

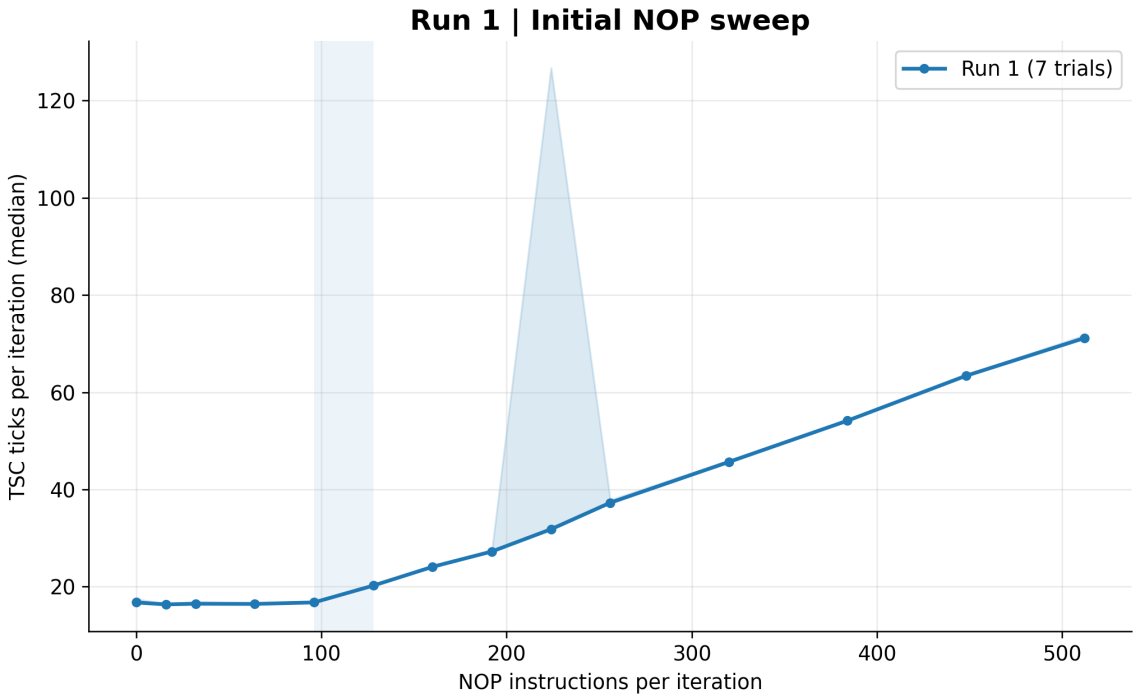


Figure 2. Run 1: median ticks per iteration versus NOP count (seven trials per count). The highlighted early-transition region and the wide IQR near 224 NOPs caution against interpreting a single slope change as ROB capacity.

Run 2 (64-byte nodes and control). In Run 2 the measured single-chain baseline was **304.58 ticks per dependent step**, much larger than the per-iteration times in Run 1. This supports the view that the effective memory-access regime changed materially; because the two runs use different layouts and workloads, their absolute timings are not two measurements of the same latency (Fig. 4). Figure 3 shows the revised two-chain timings with the NOP-only control. The control rises steadily with NOP count, so filler throughput contributes nonzero time. The two-chain curve is relatively flat in the low 300s over much of the lower sweep before increasing at larger NOP counts. That the curves differ is compatible with overlap of memory stalls and independent instruction processing; it does not by itself isolate ROB occupancy.

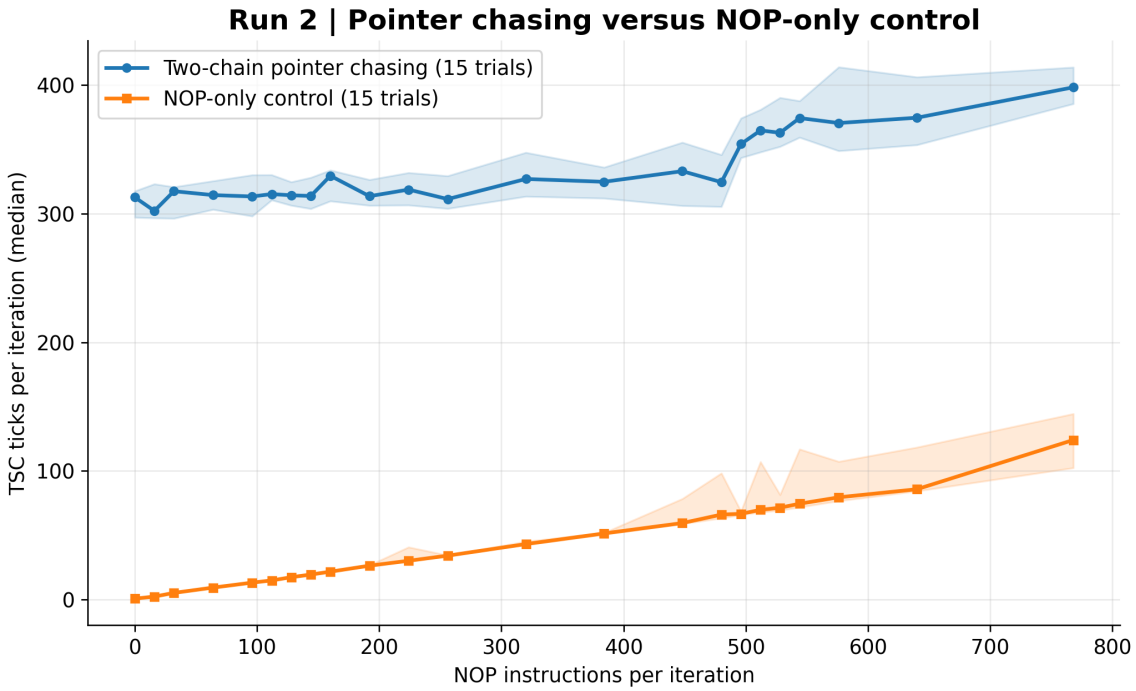


Figure 3. Run 2: two-chain pointer chasing versus the NOP-only control. Lines are medians of 15 trials; shaded areas are IQRs. The curves cannot be related by direct subtraction.

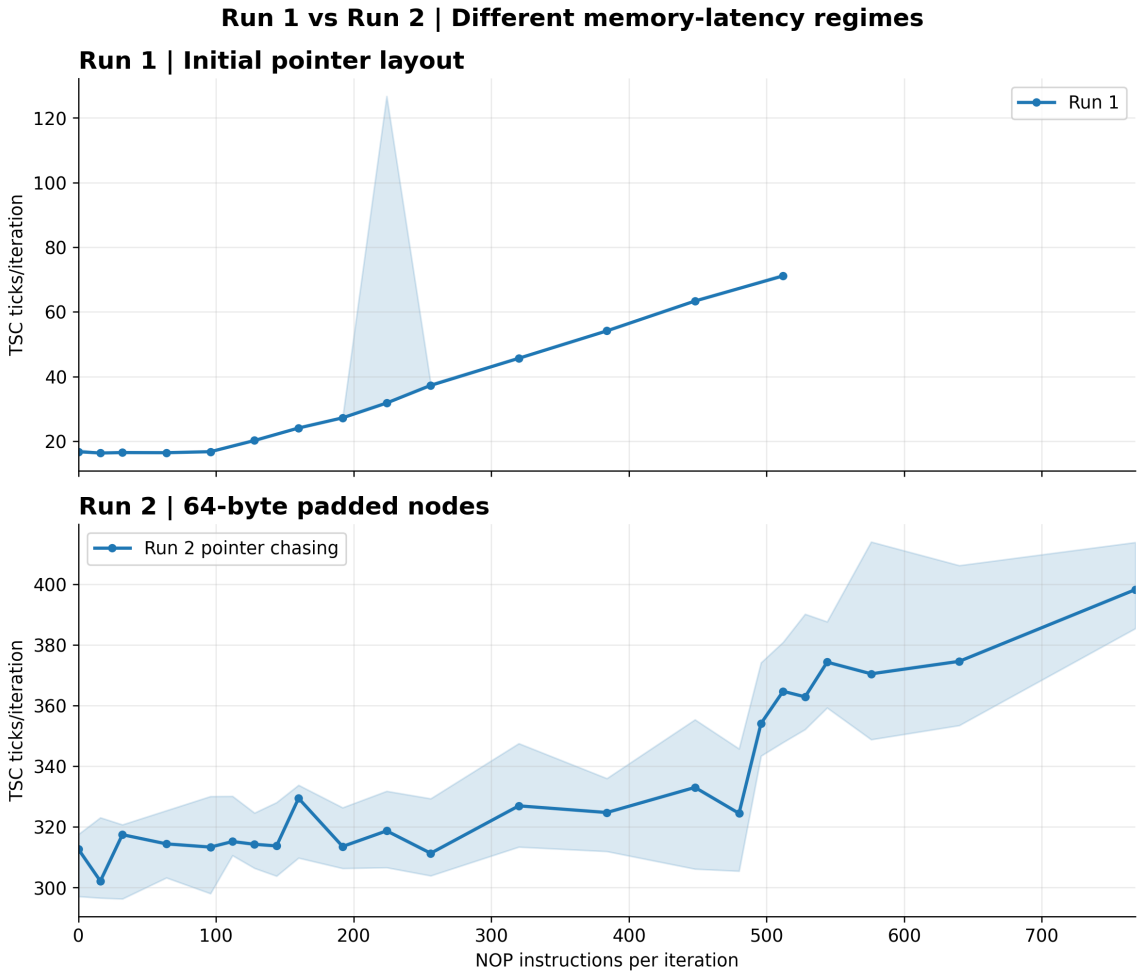


Figure 4. Run 1 and Run 2 on separate vertical scales, emphasizing that they used different memory layouts and operated in different latency regimes.

Candidate transition near 512 NOPs. Figure 5 enlarges Run 2 between 448 and 576 NOPs. The median was about 324.5 ticks at 480 NOPs, 354.1 at 496, and 364.7 at 512 (Table 5), an apparent change across 480–512. However, the point at 448 is higher than at 480, the curve is not monotonic here, and the IQR bands are substantial. Moreover, the number of in-flight micro-operations is not necessarily equal to the written NOP count: the loop, loads, and other resource limits all affect the correspondence. I therefore label this region a *candidate timing transition*, not a measurement establishing exactly 512 ROB entries.

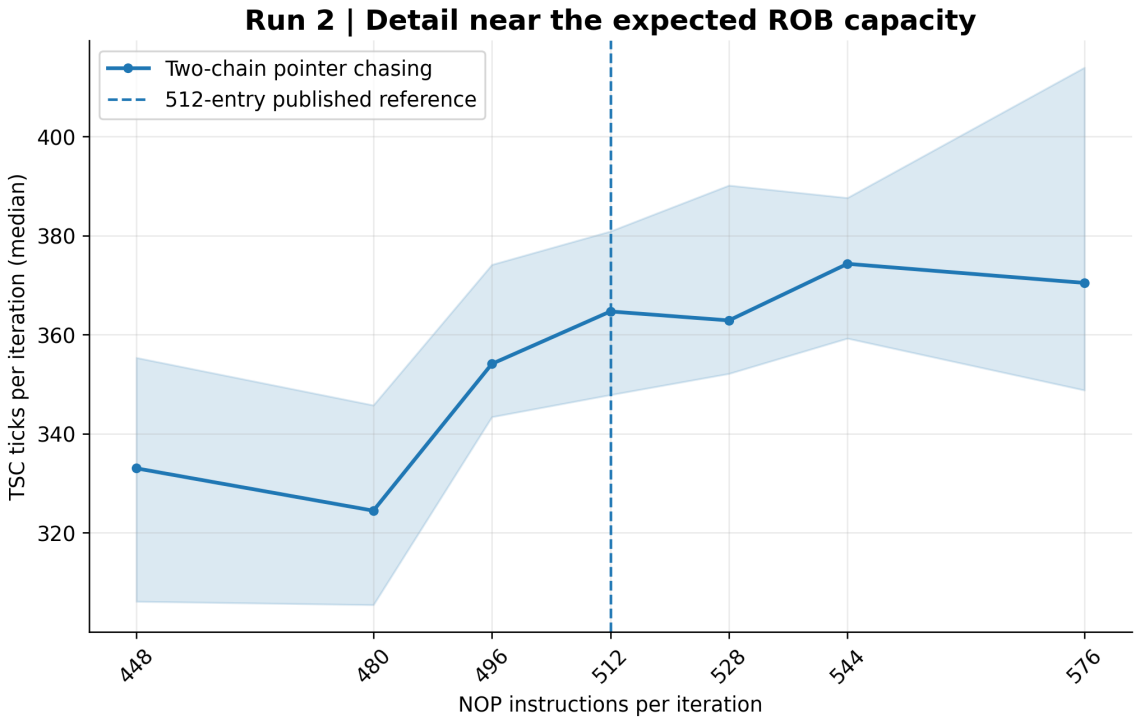


Figure 5. Run 2 detail near the architectural 512-entry reference. The vertical dashed line marks the independently published ROB size and is *not* an experimentally determined change point.

Table 5. Selected run-level observations (per-NOP medians). Times are TSC ticks per iteration.

Measurement	Run 1	Run 2 two-chain	Run 2 control
0 NOPs	16.55	312.69	0.74
128 NOPs	20.22	314.28	17.41
256 NOPs	37.28	311.33	34.17
480 NOPs	—	324.47	66.10
496 NOPs	—	354.10	66.70
512 NOPs	71.13	364.68	69.76

3.4 Native versus SGX

3.4.1 Enclave configuration and protocol

To address the Track B enclave requirement I used Intel SGX on the same machine. The processor exposed the `sgx` and `sgx_lc` flags, and Linux provided `/dev/sgx_enclave` and `/dev/sgx_provision`. I built a user-local installation of Gramine, a library OS that runs a conventional Linux executable inside an SGX enclave, and verified enclave execution with its Hello World sample [2].

Table 6. SGX configuration: what was and was not recorded.

Item	Status
Mode	Hardware SGX through Gramine (not SGX simulation)
Debug enclave	`sgx.debug = true`
Argument forwarding	`loader.insecure__use_cmdline_argv = true`
Benchmark source, compiler flags, CPU	Identical to native (`-O2`, logical CPU 10)
Per-point configuration	2^{20} 64-byte nodes per chain; 100,000 measured iterations; 10,000 warm-up; 15 trials
EPC size, enclave-size reservation, full Gramine version	**Not preserved in the logs; not asserted**
Native baseline	Previously recorded Run 2, not an interleaved paired run

```
# Compile the benchmark once (native x86-64 ELF)
gcc -O2 -Wall -Wextra -std=gnu11 -fno-unroll-loops ROB.c -o ROB
# Native mode
taskset -c 10 ./ROB > run2_results.csv
# Gramine SGX mode (after generating and signing ROB.manifest.sgx)
taskset -c 10 gramine-sgx ROB > run_sgx_results.csv
```

Listing 5. Commands for native and SGX execution on the same logical CPU.

3.4.2 Measured enclave timing

The enclave pointer-chain curve was unusually flat through 480 NOPs and showed a clear discontinuity between 480 and 496 NOPs. The median SGX latency rose from about 133.64 to 153.10 ticks per iteration, a change of 19.46 ticks (≈14.6%). Over the same interval the enclave NOP-only control changed only from about 63.49 to 65.48 ticks (≈1.99 ticks). Above 496 NOPs the pointer-chasing time increased gradually. The result is based on medians of 15 trials. The first zero-NOP trial and part of the fourth sweep were notably slower than neighboring trials; medians limit their leverage but cannot remove systematic interference.

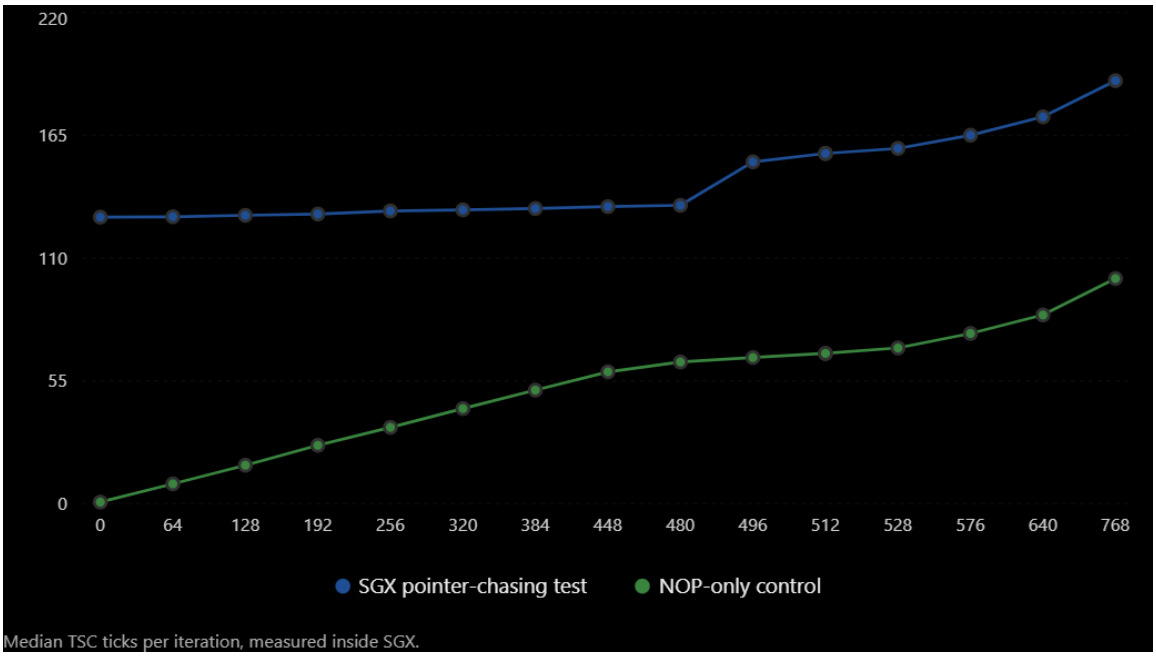


Figure 6. SGX measurement: median ticks per iteration versus filler NOP count for the two-chain benchmark (upper curve) and the NOP-only control (lower curve). The pointer-chasing discontinuity near 496 NOPs is not mirrored by the smooth control trend.

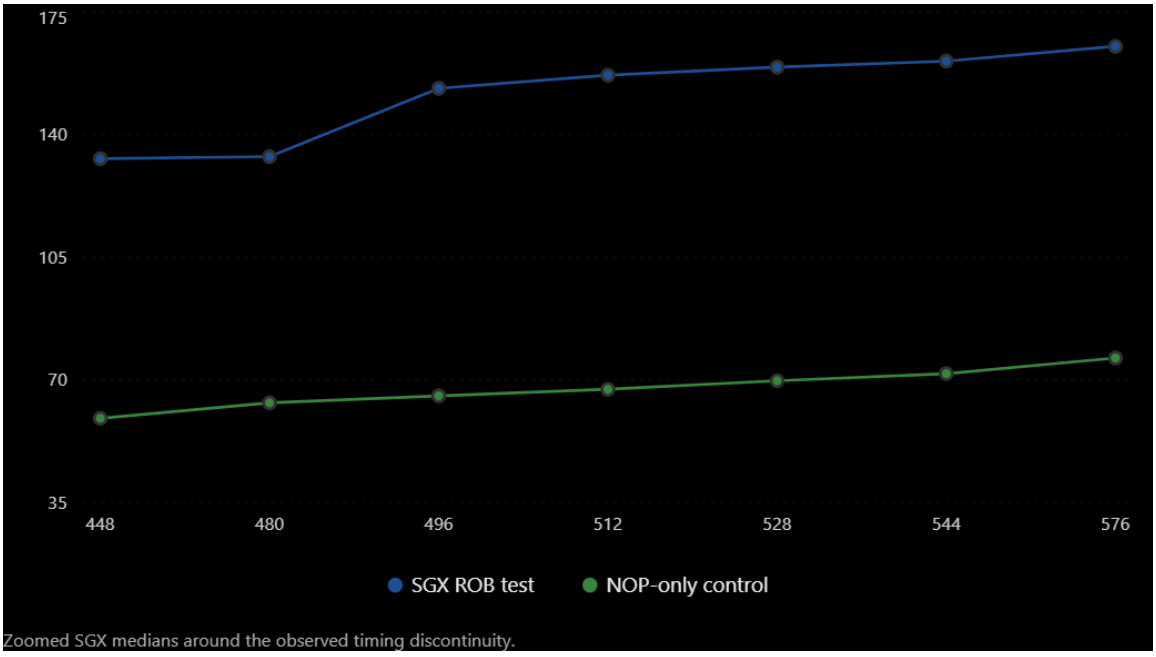


Figure 7. Zoom of the enclave measurement around the 480–496-NOP discontinuity. The points show a timing transition, not a direct observation of physical ROB occupancy.

3.4.3 Direct comparison

Table 7 and Fig. 8 compare matching NOP counts. The native Run 2 single-chain baseline was 304.58 ticks per dependent load, while the later SGX run printed 273.13 ticks per step. The two-chain median at zero NOPs was about 312.69 ticks natively and 128.37 in the enclave. The lower SGX numbers are *not* sufficient to conclude that SGX accelerates memory access or changes the ROB: the runs were separated in time, and EPC allocation or paging, cache warmness, scheduling, sibling-thread activity, and system load were not isolated. Two independent chains can also overlap loads, so two-chain iteration latency is not directly comparable to single-chain step latency.

Table 7. Native Run 2 versus SGX (medians of 15 trials; TSC ticks per two-chain iteration). The last column is the SGX NOP-only control.

NOP count	Outside SGX	Inside SGX	SGX control
0	312.69	128.37	0.74
128	314.28	129.17	17.26
256	311.33	131.13	34.20
448	333.02	133.06	59.07
480	324.47	133.64	63.49
496	354.10	153.10	65.48
512	364.68	156.87	67.33
528	362.86	159.13	69.77
576	370.44	165.05	76.25
768	398.20	189.46	100.85

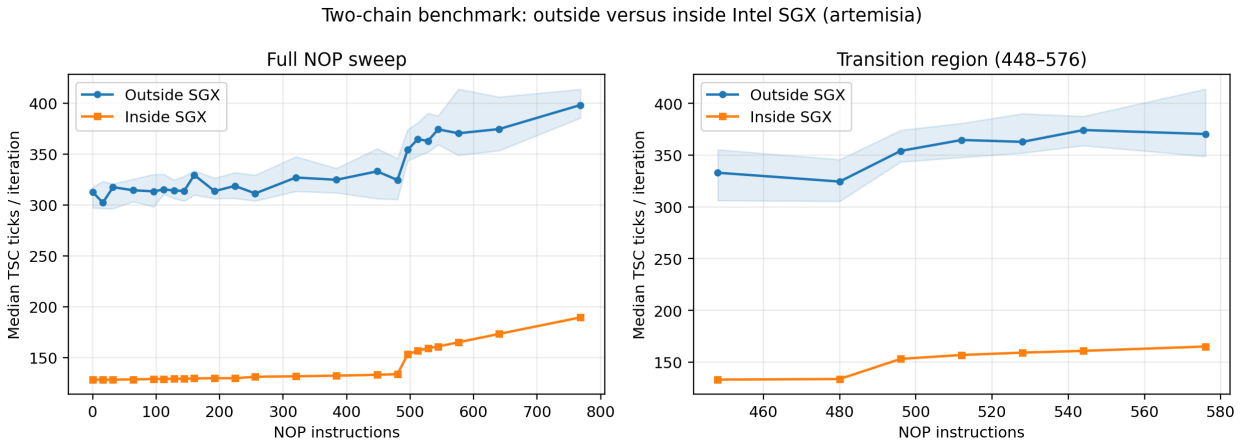


Figure 8. Native Run 2 versus SGX two-chain timing, replotted from the recorded CSV rows. Left: full sweep. Right: the 448–576-NOP region. Lines are medians; shaded envelopes are IQRs, not confidence intervals.

3.4.4 Assembly verification of the transition region

To check whether the compiler produced materially different bodies for the 480- and 496-NOP variants, I disassembled the executable with `objdump -d -M intel` and compared their non-NOP

listings. Both functions had **52 non-NOP instructions**; the static listings contained **482** and **498** NOPs, respectively. The listings therefore differ by the intended 16 NOPs, with two extra NOPs per listing likely from alignment or other emitted code. The remaining differences were mostly branch-target addresses rather than new arithmetic or memory instructions. This lowers the likelihood of an unrelated compiler rewrite, but it does not prove equal instruction-fetch alignment, page placement, micro-op-cache behavior, or in-flight ROB occupancy; the 52 instructions are a static count for the whole function, not a count of instructions simultaneously in the ROB.

3.5 Challenges and limitations

1. **Naive threshold at 96–128 NOPs.** *Observation:* Run 1 produced an early, gradually increasing curve. *Diagnosis:* its short iteration time made filler-processing overhead a credible confounder. *Response:* I moved to 64-byte aligned nodes and added a control. *Outcome:* the single-chain baseline became 304.58 ticks per step and the two-chain behavior changed substantially, supporting a change of memory regime (not a proof of DRAM misses).

2. **Fillers consume time.** *Observation:* the Run 2 control rose from about 0.74 ticks at zero NOPs to about 69.76 ticks at 512 NOPs. *Response:* I report both curves together instead of attributing every increase to the ROB. *Remaining issue:* control and two-chain times are not additive; a front-end sensitivity test would improve attribution.

3. **Instability and outliers.** Both runs had unusually long trials and Run 2 varied at some NOP counts. I raised the trial count from 7 to 15, used medians and IQRs, and alternated sweep direction. CPU frequency changes, scheduling interference, and sibling-thread activity were not explicitly controlled.

4. **Instruction count versus ROB entries.** A fixed number of explicit `nop` instructions does not automatically equal that many occupied ROB entries, because loop micro-operations, special NOP handling, and other resource limits intervene. The `objdump` check (above) verifies a broadly matching instruction structure but not identical alignment.

5. **No performance counters.** With `perf` prohibited, I used only TSC timing, controlled assembly sequences, and software controls.

SGX-specific caveats

SGX runs the protected process on the same out-of-order hardware as native code; it does not create an enclave-specific ROB capacity. Timing differences can stem from protected-memory translation, EPC contention or paging, data placement, OS interference, or the timing environment. The two 64-MiB chains occupy about 128 MiB before Gramine overhead, and EPC capacity was not measured. The exact enclave semantics of `RDTSC/RDTSCP` on this platform were not independently calibrated, and native and enclave data were not collected as randomized paired blocks. The large difference in absolute timing is therefore an observation under two configurations, not a causal SGX speedup.

3.6 Capacity estimate and prediction

My working prediction for `artemisia` is a **512-entry ROB**, consistent with public descriptions of Golden Cove [4, 5]. The experimental support is a reproducible timing discontinuity between 480

and 496 inserted NOPs in SGX, a weaker corresponding change in the native data, and the absence of a comparable discontinuity in the NOP-only control. Because each timed iteration also contains pointer loads and loop instructions, the NOP count is not an exact measure of concurrent ROB occupancy, so this remains a prediction rather than an established capacity.

Falsifiable prediction

If the transition is principally a ROB-allocation limit, its location should stay near the same *effective instruction-window occupancy* when code alignment is varied and memory stalls remain long. If instead it moves markedly with instruction placement or with the NOP-only throughput curve, a front-end or code-layout explanation becomes more plausible.

4 Question 5: Physical Register File (PRF) Characterization

4.1 Hypothesis and design

The physical register file holds renamed versions of architecturally visible registers. A long-latency operation can delay retirement while younger register-producing instructions allocate new destination versions. If the pool of a register class is exhausted before a later independent load can be renamed, overlap between the two memory chains declines. This is an indirect timing test: a rename-resource limit may change latency, but a latency change alone is not proof of the PRF's total size.

I tested four filler classes: NOPs (no register destination; reference), integer register-to-register `add` (GPR pressure), packed 128-bit `paddd` (XMM/vector pressure), and a mixed sequence of integer and vector operations (Table 8). Each class used the same two 64-byte-node chains as the revised ROB benchmark.

Table 8. PRF filler classes. Fillers rotate over eight destinations to avoid one long dependence chain; `r15` and `xmm8` are never written, so the fillers do not depend on the load.

Class	Filler	Resource stressed
`nop`	`nop`	ROB entry only (reference)
`gpr`	`add r15, rN` (register-register)	Integer rename registers (and flags)
`simd`	`paddd xmm8, xmmN`	128-bit vector registers
`mix`	4 `add` + 4 `paddd`, interleaved	Whether integer and vector pools are shared

Register-register adds were chosen over add-immediate because some cores can fold small immediates at rename, which would hide the allocation. Listing 6 shows the filler definitions. Each full unit contains eight instructions, and a 4-instruction tail provides 4-instruction resolution.

```
#define UNIT_GPR \
    "add %%r15, %%r8\n\t"  "add %%r15, %%r9\n\t"  \
    "add %%r15, %%r10\n\t" "add %%r15, %%r11\n\t" \
    "add %%r15, %%r12\n\t" "add %%r15, %%r13\n\t" \
    "add %%r15, %%r14\n\t" "add %%r15, %%rax\n\t"

#define UNIT_SIMD \
    "paddd %%xmm8, %%xmm0\n\t" "paddd %%xmm8, %%xmm1\n\t" \
    "paddd %%xmm8, %%xmm2\n\t" "paddd %%xmm8, %%xmm3\n\t" \
    "paddd %%xmm8, %%xmm4\n\t" "paddd %%xmm8, %%xmm5\n\t" \
    "paddd %%xmm8, %%xmm6\n\t" "paddd %%xmm8, %%xmm7\n\t"

```

```
#define UNIT_MIX \
    "add %%r15, %%r8\n\t"  "paddd %%xmm8, %%xmm0\n\t" \
    "add %%r15, %%r9\n\t"  "paddd %%xmm8, %%xmm1\n\t" \
    "add %%r15, %%r10\n\t" "paddd %%xmm8, %%xmm2\n\t" \
    "add %%r15, %%r11\n\t" "paddd %%xmm8, %%xmm3\n\t"

#define TAIL_GPR  /* 4-instruction tail */ \
    "add %%r15, %%r8\n\t"  "add %%r15, %%r9\n\t"  \
    "add %%r15, %%r10\n\t" "add %%r15, %%r11\n\t"
```

Listing 6. Register-class-specific filler units (`PRF.c`). A full unit is eight instructions; a tail is four.

The generator (Listing 7) places a load from chain A, $8R + 4T$ fillers of one class, and a load from chain B inside the timed loop. A single clobber list is shared by every class so the compiler allocates the loop's own registers identically across all tests.

```
// Same clobber list for every class: identical loop register allocation.
#define CLOBBERS \
    "rax", "r8", "r9", "r10", "r11", "r12", "r13", "r14", "r15", \
    "xmm0", "xmm1", "xmm2", "xmm3", "xmm4", "xmm5", "xmm6", "xmm7", \
    "xmm8", "memory", "cc"

#define DEFINE_PRF_TEST(CLS, UNIT, TAIL, R, T)                          \
static __attribute__((noinline))                                        \
double CLS##_test_##R##_##T(Struct1 *a, Struct1 *b, uint64_t steps) {   \
    uint64_t t0 = x86_tsc_start();                                      \
    for (uint64_t i = 0; i < steps; i++) {                              \
        __asm__ volatile (                                              \
            "mov (%0), %0\n\t"                      /* load A */        \
            ".rept " STRINGIFY(R) "\n\t" UNIT ".endr\n\t"  /* 8*R fillers */\
            ".rept " STRINGIFY(T) "\n\t" TAIL ".endr\n\t"  /* 4*T fillers */\
            "mov (%1), %1\n\t"                      /* load B */        \
            : "+r"(a), "+r"(b) : : CLOBBERS);                           \
    }                                                                   \
    uint64_t t1 = x86_tsc_stop();                                       \
    sink_a = a;  sink_b = b;                                            \
    return (double)(t1 - t0) / steps;                                   \
}
```

Listing 7. PRF probe generator and shared clobber list (`PRF.c`).

Sweep and trials. The current `PRF.c` specifies 21 trials, 100,000 measured iterations per trial, and a 4-instruction sweep resolution between 128 and 320 instructions, with coarser steps elsewhere (0–112 in steps of 16 and 352, 384, 416, 448, 512, 576, 640). The class controls are not subtractable overheads, because execution, retirement, and memory stalls overlap.

4.2 Native observations

Figure 9 shows all four classes, the detected candidate transitions, and the no-load controls. The GPR curve changes sharply near 212 filler instructions and the XMM `paddd` curve near 244. The mixed curve has no comparably early transition and changes at a much larger count, near the NOP-reference transition around 448. This is consistent with distinct pressure mechanisms for GPR- and SIMD-producing instructions, but it is not a unique proof of the physical register pool configuration.

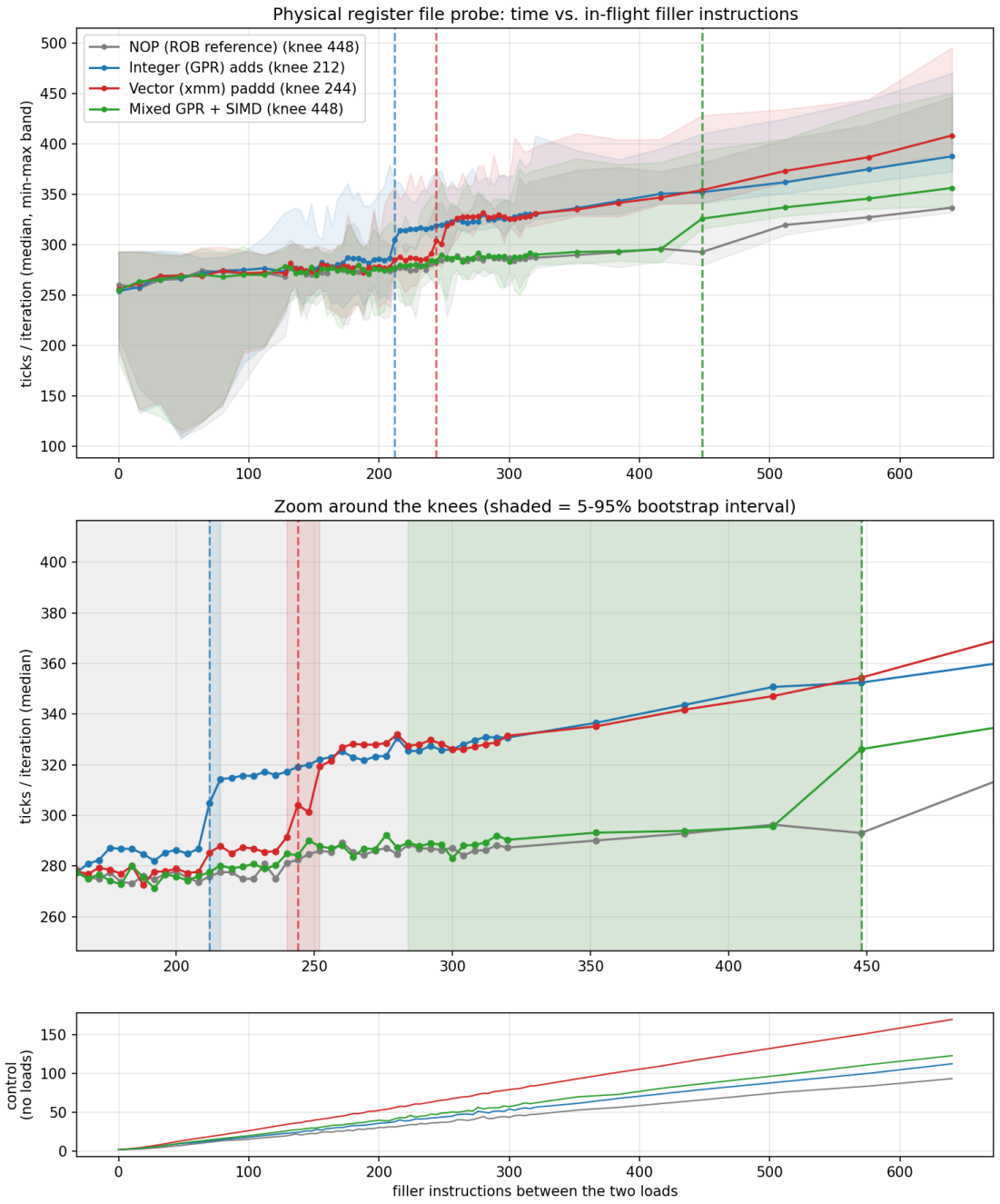


Figure 9. Native PRF experiment. Timing versus filler instructions for NOP, GPR-add, XMM-`paddd`, and mixed workloads. Top: median latency with a min–max trial band. Middle: detected candidate changes with bootstrap resampling ranges. Bottom: no-load controls. A detected "knee" is an instruction-count threshold, not directly a count of physical registers.

Table 9. Candidate native timing transitions read from the PRF figure. "Count" is injected filler instructions, not a confirmed register-file capacity. The final column reflects the sweep grid at that location.

Filler class	Count	Scope of inference	Grid spacing
Integer `add` (GPR)	212	GPR, flags, and throughput sensitive	4 instr.
XMM `paddd` (SIMD)	244	128-bit vector class	4 instr.
Mixed GPR + XMM	448	Mixed resource limit	coarse (32–64)
NOP reference	448	Reference instruction-window limit	coarse (32–64)

4.3 Enclave results and comparison

I also ran the PRF benchmark through Gramine SGX on `artemisia` with the same CPU affinity. The enclave run emitted measurement rows interleaved with diagnostic text, so some CSV records were split or malformed. The SGX medians in Table 10 are transcribed from the per-instruction-count summary that was displayed afterwards. The complete raw enclave CSV is not among the artifacts, so I do not claim to have revalidated every recovered record or a formal confidence interval for its thresholds.

Table 10. Selected SGX PRF medians from the terminal output (TSC ticks per two-chain iteration). Rows examine the candidate integer and XMM transitions and are not an exhaustive sweep.

Filler count	GPR	XMM	Mixed	NOP
208	282.16	283.77	284.01	270.45
212	313.17	290.94	288.08	285.70
216	356.94	309.54	299.98	278.12
240	367.86	340.57	322.89	318.97
244	368.80	324.58	315.02	315.75
248	365.35	340.36	310.98	312.46
252	370.34	356.71	306.95	314.81
256	360.09	358.75	303.86	305.10
448	407.64	411.69	379.95	333.57

Between 212 and 216 fillers the SGX GPR median rose from 313.17 to 356.94 ticks per iteration, while the SGX NOP reference at 216 was only 278.12. SIMD latency rose more broadly through 240–256; for example, the XMM median increased from 340.36 at 248 to 356.71 at 252. These neighborhoods broadly correspond to the native class-dependent transitions, but the SGX baseline and individual measurements are noisier. Neither the PRF's hardware capacity nor the existence of a unified physical file can be inferred by adding architectural register counts to an observed threshold.

Threats to validity (PRF)

- **Flags.** x86-64 `add` modifies condition flags as well as its destination, so the GPR threshold may reflect a flags-renaming resource rather than only integer physical registers.
- **Register reuse and throughput.** Destinations are reused; repeated renaming creates multiple physical versions, but the benchmark still contains dependence chains, finite

arithmetic throughput, ROB pressure, and possible scheduler constraints.

- **Scope.** `paddd` on XMM registers cannot establish the capacity of all vector widths or register classes, and the mixed test shares these confounders.

The measurements support *class-sensitive effective resource limits*; the exact integer and vector PRF sizes remain unresolved.

5 Question 6: Data TLB Hierarchy and Capacity

5.1 Translation-sensitive methodology

The TLB caches virtual-to-physical translations. To create translation pressure without arbitrary memory access, I built a randomized circular pointer chain containing exactly one dependent load from each of N distinct virtual pages (Fig. 10). Varying N changes how many translations a full traversal needs. A second chain of the same length uses cache-line-sized nodes packed contiguously, which reduces its translation footprint. Here each requested page was 4,096 bytes; I ran nine timed trials per page count and summarize them with medians and IQRs.

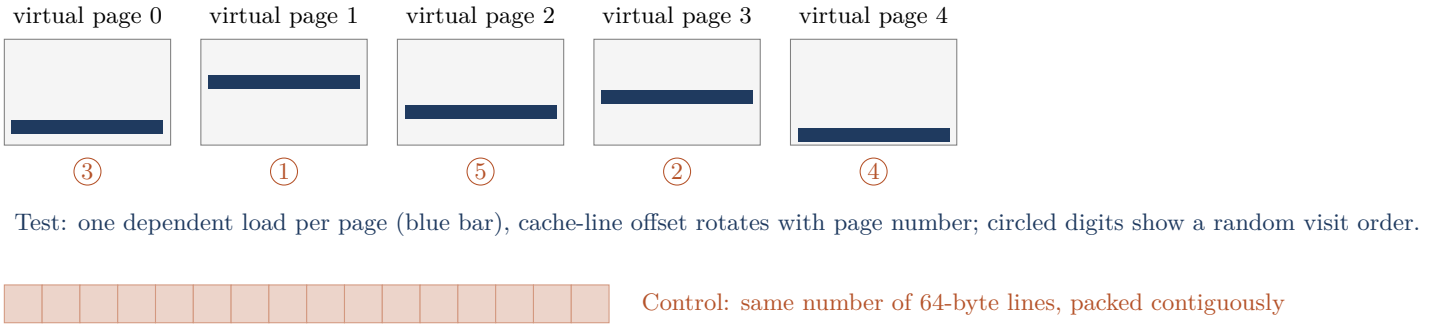


Figure 10. Memory layout of the TLB probe. The test touches one cache line in each of N pages, so each access needs a distinct translation. The control chases the same number of lines packed into few pages.

Listing 8 shows the node placement and the timed chase. The cache-line offset within each page rotates with the page number so that nodes spread over all L1D sets instead of aliasing to one.

```
static inline void **test_node(char *base, size_t page_bytes, size_t p) {
    /* Rotate the offset to avoid all nodes sharing the same L1D set. */
    return (void **)(base + p * page_bytes + (p % 64) * CACHELINE);
}
static inline void **control_node(char *base, size_t p) {
    return (void **)(base + p * CACHELINE);       /* packed lines */
}

static void build_cycle(char *base, size_t page_bytes, size_t n,
                        uint32_t seed, int control) {
    size_t *order = checked_malloc(n * sizeof(*order));
    for (size_t i = 0; i < n; ++i) order[i] = i;
    if (n > 1) shuffle(order, n, seed);
    for (size_t i = 0; i < n; ++i) {
        size_t cur = order[i], next = order[(i + 1) % n];
        void **at = control ? control_node(base, cur) : test_node(base, page_bytes, cur);
        void **to = control ? control_node(base, next) : test_node(base, page_bytes, next);
        *at = to;
    }
```

```
    free(order);
}

static __attribute__((noinline)) double chase(void *p, uint64_t steps) {
    uint64_t begin = tsc_start();
    for (uint64_t i = 0; i < steps; i += 8) {
        p = *(void *volatile *)p;     /* each load's address depends on the */
        p = *(void *volatile *)p;     /* previous load's value; eight per   */
        p = *(void *volatile *)p;     /* iteration (shown abridged: the     */
        p = *(void *volatile *)p;     /* source repeats this line 8 times)  */
    }
    uint64_t end = tsc_stop();
    sink = p;
    return (double)(end - begin) / (double)steps;
}
```

Listing 8. Node placement, cycle construction, and the timed chase (`TLB_improved.c`).

5.2 Memory setup and huge-page control

If Transparent Huge Pages (THP) backed the test region, one TLB entry would cover 2 MiB, so the 4-KiB experiments must prevent it. The test mapping is 2-MiB aligned, advised with `MADV_NOHUGEPAGE` in 4-KiB mode, and page-faulted before timing (Listing 9). The program also reads the `AnonHugePages` field of *that specific mapping* from `/proc/self/smaps` and warns if the backing is not as intended.

```
char *test = map_aligned(test_bytes, PAGE_2M);
char *ctrl = map_aligned(ctrl_map_bytes, PAGE_2M);

int advice = page_bytes == PAGE_4K ? MADV_NOHUGEPAGE : MADV_HUGEPAGE;
if (madvise(test, test_bytes, advice))
    fprintf(stderr, "WARNING: test madvise failed: %s\n", strerror(errno));
if (madvise(ctrl, ctrl_map_bytes, MADV_HUGEPAGE))
    fprintf(stderr, "WARNING: control madvise failed: %s\n", strerror(errno));

/* Force physical backing so the timed chase does not fault in pages. */
for (size_t i = 0; i < max_pages; ++i)
    test[i * page_bytes] = 0;
for (size_t i = 0; i < ctrl_map_bytes; i += PAGE_4K)
    ctrl[i] = 0;

long huge_kb = mapping_anon_huge_kb(test);   /* -1 if smaps is unavailable */
if (page_bytes == PAGE_4K && huge_kb)
    fprintf(stderr, "WARNING: 4K test appears to contain huge-page backing.\n");
```

Listing 9. Mapping, THP advice, prefaulting, and backing check (`TLB_improved.c`).

5.3 Sweeps and platforms

The benchmark accepts a page size, a maximum page count, and an optional linear window. The sweep list is geometric (about 12 points per doubling, Listing 10) for the full scan, and linear for focused scans (Table 11).

```
static size_t *geometric_sweep(size_t max_pages, size_t *out_n) {
    ...
    double x = 1.0;
    while (x <= (double)max_pages + 0.49) {
        size_t p = (size_t)(x + 0.5);
        if (p > max_pages) break;
        if (p > previous) { list[n++] = p; previous = p; }
```

```
        x *= 1.0594630943592953;     /* 2^(1/12): ~12 points per doubling */
    }
    ...
}
```

Listing 10. Geometric sweep generation (`TLB_improved.c`, abridged).

Table 11. TLB sweeps (4-KiB pages; nine trials per point).

Sweep	Range and spacing	Purpose
Full	1–16,384 pages, geometric (≈12 points per doubling)	Overview
L1-focused	48–160 pages, step 4	First transition
L2-focused	1,536–3,072 pages, step 64	Second-level region

Native and Gramine SGX runs used the same machine and logical CPU 10. The enclave experiments used a Gramine *debug enclave* (`sgx.debug = true`) with host-argument forwarding for benchmark parameters [7]. Gramine could not expose per-mapping `AnonHugePages` inside the enclave; the log reported `test AnonHugePages=unavailable`, so equal page-backing conditions between the modes were not independently confirmed.

5.4 First-level DTLB transition

Figure 11 compares the narrow first-transition region at identical page counts. Both curves stay near 3.7 ticks per dependent load through about 96 pages. The native curve then rises rapidly to roughly 8.8–8.9 ticks, while the enclave curve rises more gradually to about 9.0 ticks and converges at larger working sets. The packed-lines control, absent from the comparison plot but recorded separately, stayed near the low baseline over this interval in the native run. The shared transition supports an *effective first-level 4-KiB data-TLB capacity near 96 entries*, i.e. $96 \times 4\,\mathrm{KiB} = 384\,\mathrm{KiB}$ of virtual-page coverage. The exact boundary is bounded by the sweep grid and may depend on conflicts and concurrency.

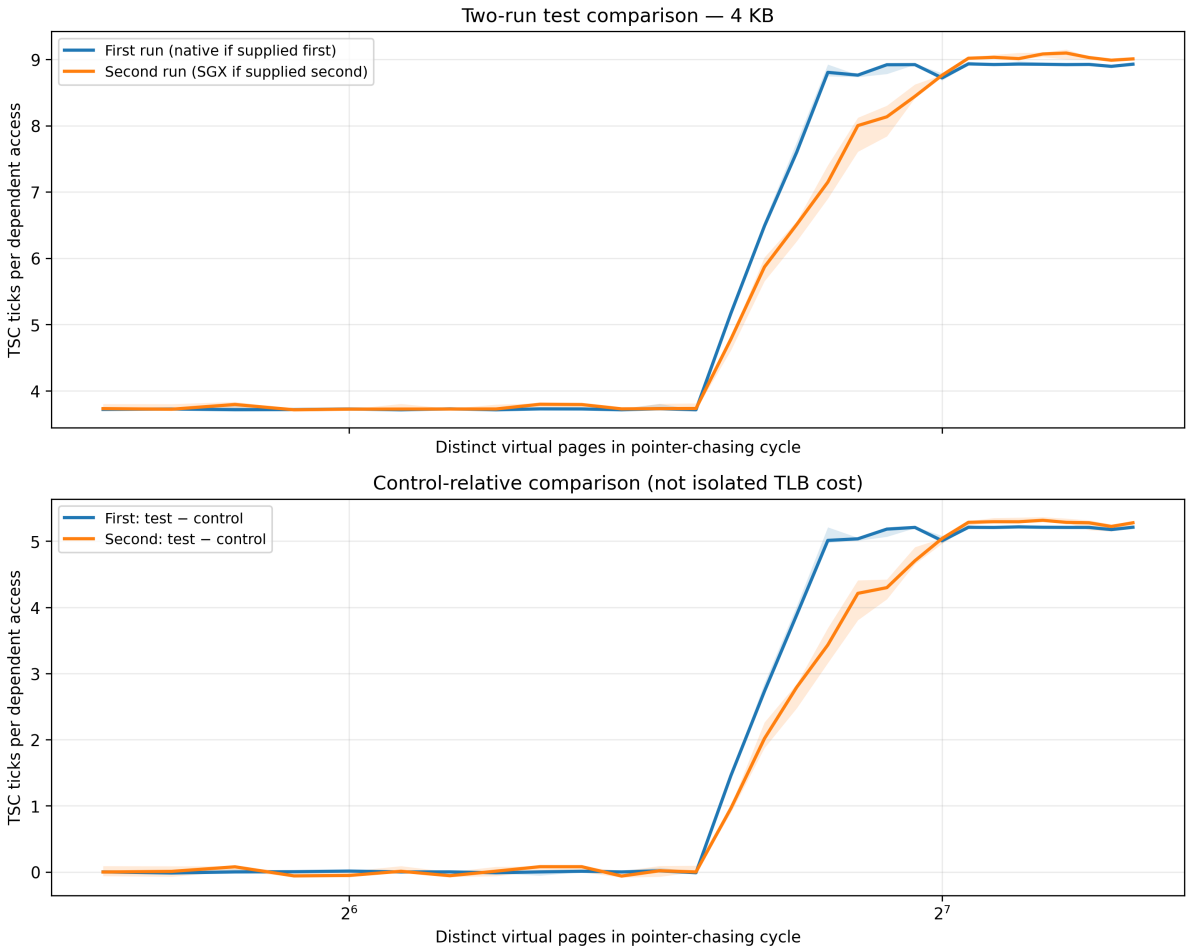


Figure 11. L1 DTLB-focused sweep, 48–160 distinct 4-KiB pages. Blue: native Linux. Orange: Gramine SGX. Upper panel: median latency with IQR bands. Lower panel: each run's test minus its packed-lines control. Both curves transition near 96 pages; the native rise is sharper.

5.5 Second-level translation pressure

Figure 12 spans 1,536–3,072 pages. Native latency increases gradually from about 18–21 ticks per access at the low end to roughly 28 at the high end. Inside SGX it rises more strongly, reaching about 49–50 ticks by 3,072 pages. Near 2,048 pages the figure suggests roughly 21 ticks natively and 29 in SGX; these are visual approximations from the plot, not re-extracted raw medians. The control-relative difference also grows much more under SGX.

A candidate second-level TLB of about 2,048 four-KiB translations would cover $2048 \times 4\,\text{KiB} = 8\,\text{MiB}$, consistent with a published Golden Cove STLB reference [1]. The observed knee is broad rather than pinned to 2,048, and page-walk caches, translation-set conflicts, cache effects, enclave backing, and EPC-related behavior remain plausible contributors.

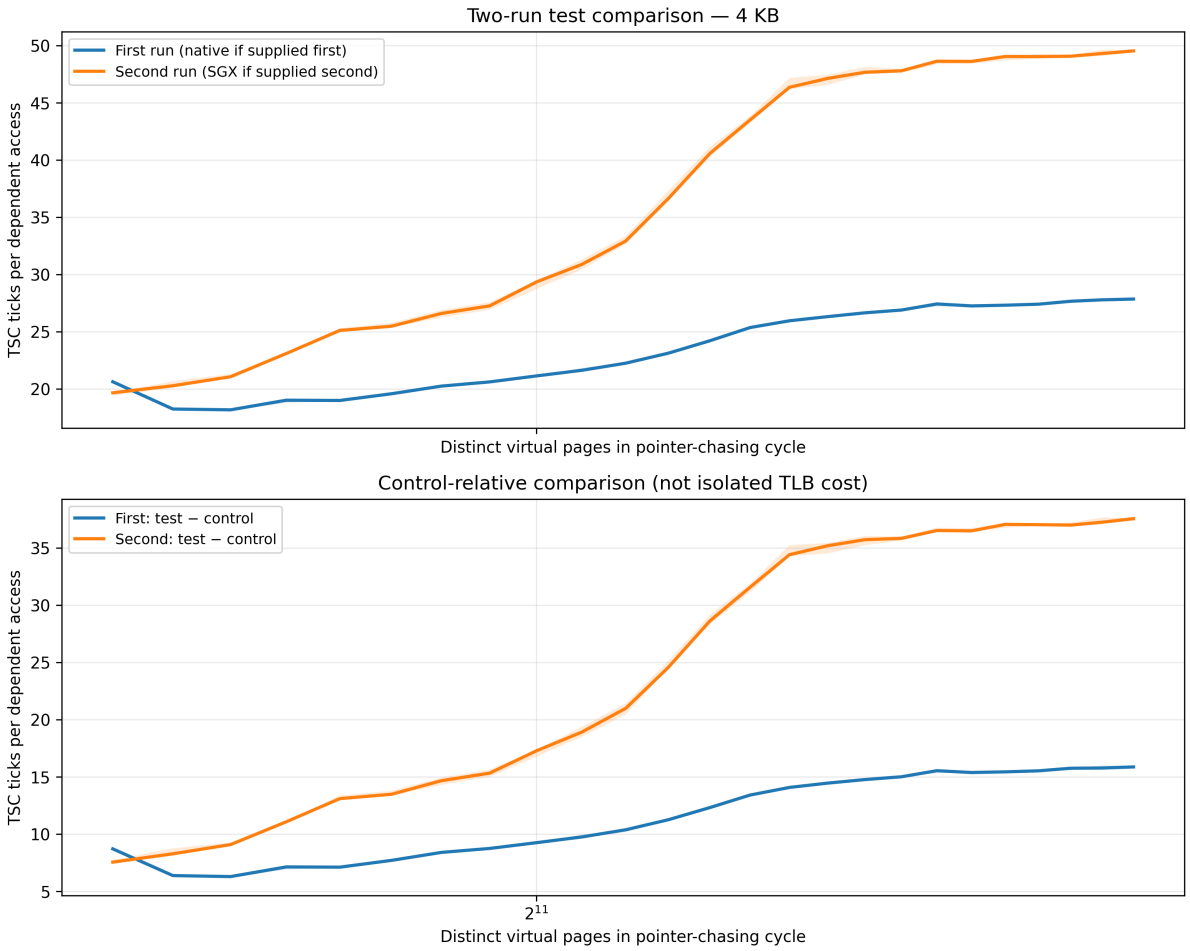


Figure 12. L2/STLB-focused sweep, 1,536–3,072 distinct 4-KiB pages. Native (blue) and SGX (orange) medians diverge at larger working sets, with markedly higher enclave-side latency. Shading denotes IQRs, not confidence intervals. The broad transition cannot uniquely establish the STLB entry count.

5.6 Full sweep and the page-walk regime

Figure 13 gives the full native-versus-enclave comparison. Both modes share a low-latency baseline for small working sets and a similar first translation-sensitive threshold, then diverge: the enclave becomes substantially slower over an intermediate range before native timing rises strongly at still larger counts. This nonconstant separation argues against treating SGX as a fixed per-access penalty, and cautions against interpreting every step as another hardware TLB level, since large-working-set costs also involve page walks, page-table cache misses, and the data-cache hierarchy.

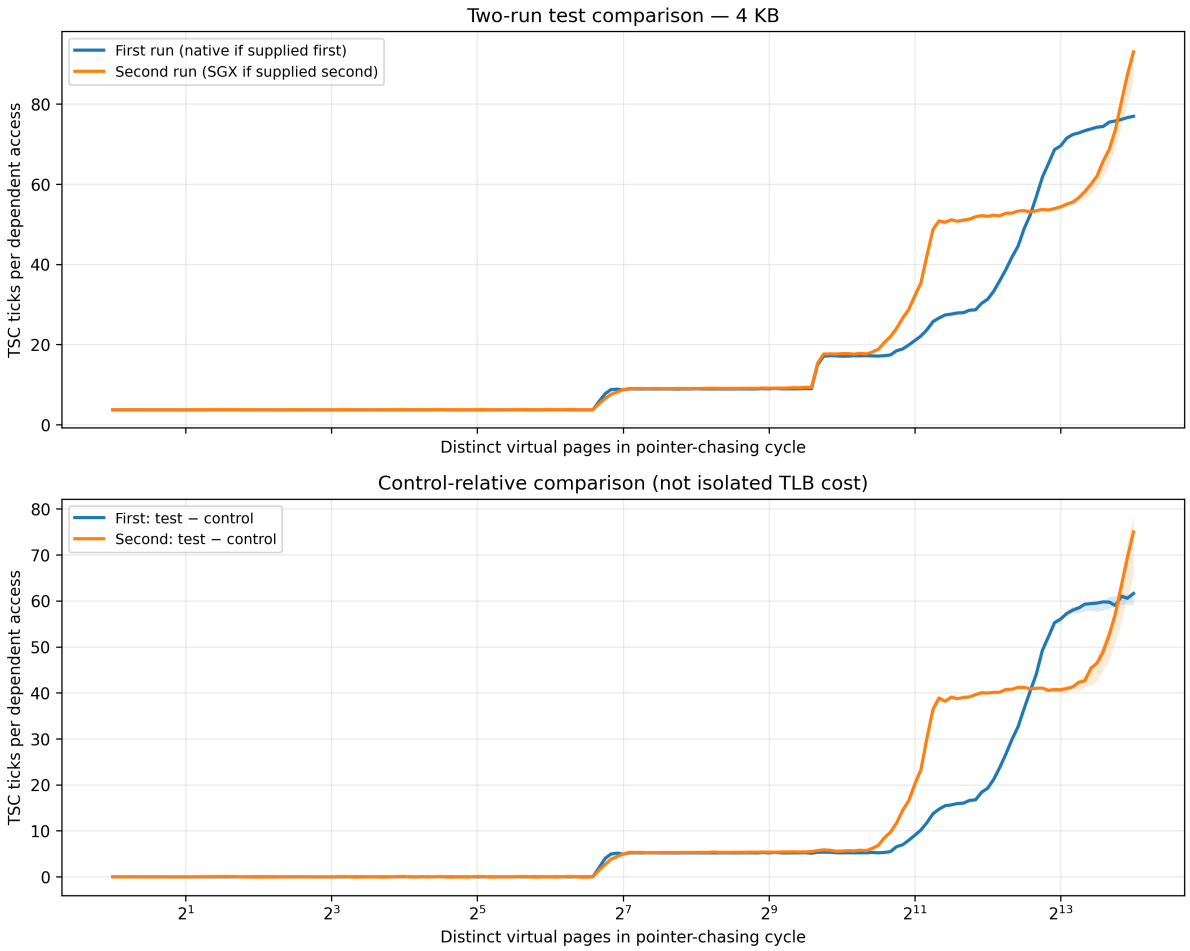


Figure 13. Full TLB-sensitive sweep, native versus Gramine SGX. Medians and IQRs over a logarithmic page-count axis (blue: native; orange: SGX). The lower panel is each run's difference from its own packed-lines control, which is not an isolated TLB-miss latency.

Table 12. Data-TLB interpretation of the 4-KiB experiments. Hardware references are distinct from observed latency thresholds.

Property	Directly observed	Qualified interpretation
First transition	Both modes rise in the vicinity of 96 pages	Effective L1 DTLB capacity near 96 entries; stronger evidence
Larger working sets	Native rises gradually; SGX rises more steeply across 1,536–3,072 pages	Second-level TLB near 2,048 entries is plausible but not independently resolved
Later large-set behavior	Additional strong increases in the full sweep	A further TLB level cannot be identified without separating page walks from caches
Page-size verification	Native test reported no huge-page backing; SGX reported backing information unavailable	The nominal 4-KiB mode is intended, but backing equivalence cannot be fully verified

Limitations (TLB)

- **Control is not TLB-free.** The packed-line control has the same number of loads but not the same physical cache-set mapping, page-table footprint, prefetch behavior, or memory residence as the page-spaced test. In the default 16,384-page sweep the control occupies $16{,}384 \times 64$ bytes $= 1$ MiB, i.e. 256 ordinary 4-KiB pages; a mapping smaller than 2 MiB cannot be backed by a huge page, so at the largest counts the control may itself incur first-level DTLB misses and "test minus control" can under-report the translation cost.
- **Attribution.** Timing increases near 2,000 pages therefore cannot be assigned entirely to a second-level TLB miss.
- **Grid and associativity.** The page-count sweep samples a finite grid, and set associativity can make an effective threshold smaller than an architectural entry count.
- **Platform.** The SGX and host tests share the physical processor but not necessarily identical memory-management behavior. Only the *data* TLB is characterized; the instruction TLB is not.

TLB prediction

I predict two relevant levels of data-side translation caching: an **L1 DTLB of about 96 entries** and a larger **STLB plausibly near 2,048 entries**, with substantially greater confidence in the first estimate.

6 Cross-Experiment Interpretation

Across all three tests, the enclave is not a different microprocessor. Differences between timing curves can arise from memory placement, SGX translation and backing behavior, data residency, platform scheduling, and measurement variability even when the underlying resource sizes are unchanged. I therefore treat reproducible transitions as evidence for resource-capacity hypotheses, not as automatic capacity readouts.

Table 13. Summary of selected Track B exercises. ROB and PRF thresholds count inserted instructions; TLB thresholds count distinct 4-KiB pages.

Exercise	Experimental observation	Supported claim
Q4: ROB (30 pts)	SGX jump at 480–496 inserted NOPs; weaker native change	A 512-entry reference is plausible, but exact ROB capacity was not isolated
Q5: PRF (35 pts)	Native class-dependent knees near 212 GPR and 244 XMM fillers; SGX in similar neighborhoods	Distinct effective, register-class-sensitive constraints; exact PRF size unresolved
Q6: TLB (35 pts)	Shared first transition near 96 pages; large-set SGX latency higher than native	Consistent with an L1 DTLB near 96 entries and an STLB perhaps near 2,048 entries

7 Conclusion

I completed the three selected undergraduate Track B questions on one course machine, `artemisia`, and compared behavior with Intel SGX where supported. The ROB experiments evolved from a naive compact pointer-chain layout to a controlled two-chain probe with a NOP-only reference, yielding a strong enclave transition near 480–496 NOPs that is consistent with, but not definitive proof of, a 512-entry ROB. The PRF experiment revealed earlier, class-dependent thresholds for integer and XMM-producing instructions, but flags renaming and instruction throughput prevent a clean inference of total physical-register counts. The 4-KiB DTLB experiment gave its clearest evidence near 96 active pages in both native and SGX execution; the broader second-level transition is compatible with about 2,048 entries at lower confidence. The most important result of the project is a controlled, repeatable set of measurements together with an explicit distinction between what was observed and what is inferred.

A Additional Code: Measurement Drivers

The listings below show the loops that sweep the experiments and write CSV rows. All three alternate sweep direction and the test/control order between trials to reduce ordering bias, and all print one CSV row per measurement.

```
for (int trial = 1; trial <= TRIALS; trial++) {
    for (size_t k = 0; k < num_tests; k++) {
        // Alternate NOP sweep direction each trial
        size_t t = (trial % 2 == 1) ? k : num_tests - 1 - k;
        int nops = tests[t].nops;
        double rob_ticks, control_ticks;

        // Alternate which experiment runs first
        if (trial % 2 == 1) {
            rob_ticks     = tests[t].rob_function(chain_a, chain_b, STEPS);
            control_ticks = tests[t].control_function(STEPS);
        } else {
            control_ticks = tests[t].control_function(STEPS);
            rob_ticks     = tests[t].rob_function(chain_a, chain_b, STEPS);
        }
        printf("rob,%d,%d,%.4f\n",     nops, trial, rob_ticks);
        printf("control,%d,%d,%.4f\n", nops, trial, control_ticks);
    }
}
```

Listing 11. ROB driver (`ROB.c`, condensed).

```
/* Sweep points X(R,T) -> 8*R + 4*T filler instructions (abridged):
     X(0,0) X(2,0) ... X(14,0)        0..112, step 16
     X(16,0) X(16,1) ... X(39,1)      128..316, step 4
     X(40,0)                          320
     X(44,0) X(48,0) X(52,0) X(56,0) X(64,0) X(72,0) X(80,0)   352..640 */

for (int trial = 1; trial <= TRIALS; trial++) {
    for (size_t k = 0; k < num_tests; k++) {
        size_t t = (trial % 2 == 1) ? k : num_tests - 1 - k;
        for (int ci = 0; ci < NUM_CLASSES; ci++) {
            int c = (trial % 2 == 1) ? ci : NUM_CLASSES - 1 - ci;
            double rt, ct;
            if (trial % 2 == 1) {
                rt = tests[t].test[c](chain_a, chain_b, STEPS);
                ct = tests[t].control[c](STEPS);
            } else {
                ct = tests[t].control[c](STEPS);
                rt = tests[t].test[c](chain_a, chain_b, STEPS);
```

```
            }
            printf("%s,test,%d,%d,%.4f\n",    class_names[c], tests[t].instrs, trial, rt);
            printf("%s,control,%d,%d,%.4f\n", class_names[c], tests[t].instrs, trial, ct);
        }
    }
}
```

Listing 12. PRF sweep definition and driver (`PRF.c`, condensed).

```
for (size_t s = 0; s < count; ++s) {
    size_t pages = sweep[s];
    build_cycle(test, page_bytes, pages, 12345u + (uint32_t)pages, 0);
    build_cycle(ctrl, page_bytes, pages,   777u + (uint32_t)pages, 1);
    void *tp = test_node(test, page_bytes, 0);
    void *cp = control_node(ctrl, 0);

    uint64_t warm  = round8(max(WARMUP_ACCESSES, 2 * pages));
    uint64_t steps = round8(max(MIN_ACCESSES,    8 * pages));
    (void)chase(tp, warm);
    (void)chase(cp, warm);

    for (int trial = 1; trial <= TRIALS; ++trial) {
        double t, c;
        if (trial % 2) { t = chase(tp, steps); c = chase(cp, steps); }
        else           { c = chase(cp, steps); t = chase(tp, steps); }
        printf("%zu,%zu,%d,%.4f,%.4f\n", page_bytes, pages, trial, t, c);
    }
}
```

Listing 13. TLB driver (`TLB_improved.c`, condensed; `max` abbreviates the conditional expressions in the source).

B Reproducibility and Artifact Inventory

```
gcc -O2 -Wall -Wextra -std=gnu11 -fno-unroll-loops ROB.c -o ROB
taskset -c 10 ./ROB > run2_results.csv
taskset -c 10 gramine-sgx ROB > run_sgx_results.csv

cp 'PRF(1).c' PRF.c
gcc -O2 -Wall -Wextra -std=gnu11 -fno-unroll-loops PRF.c -o PRF
taskset -c 10 ./PRF > prf_native_results.csv
taskset -c 10 gramine-sgx PRF > prf_sgx_results.csv

cp 'TLB_improved(1).c' TLB_improved.c
gcc -O2 -Wall -Wextra -std=gnu11 -fno-unroll-loops TLB_improved.c -o TLB
taskset -c 10 ./TLB 4096 16384 > tlb_native_4k.csv
taskset -c 10 gramine-sgx TLB 4096 16384 > tlb_sgx_4k.csv
# Focused sweeps (page_bytes max_pages start end step):
#   L1: 48..160 step 4      L2: 1536..3072 step 64
```

Listing 14. Build and representative run commands. Gramine manifests must be created and signed before enclave runs; the TLB manifest forwards command-line arguments in the research setup.

Table 14. Artifact inventory.

Artifact	Contents
`code/ROB.c`	ROB probe, NOP control, and driver
`code/PRF(1).c`	PRF probe with four filler classes and driver
`code/TLB_improved(1).c`	TLB probe, THP-backing check, and driver
`code/prf_analyze.py`, `code/plot_tlb(1).py`	Analysis and plotting scripts
`figures/`	ROB figures and raw ROB data; PRF and TLB figures (supplied filenames preserved)
SGX PRF median CSV	Selected SGX PRF medians transcribed from the terminal; *not* a substitute for the raw run

Submission checklist

The raw PRF and TLB run CSV files were retained on the course server but were not among the attachments used to assemble this Overleaf copy. They must be added separately to the final submission archive to satisfy the assignment's raw-data requirement.

C AI-Use Disclosure (Track B)

I used generative AI during development and refinement of benchmark code, debugging and experiment planning, data interpretation, plotting support, and preparation of this report. The numerical results identified as measured were obtained from course-server runs and my terminal output and figures; benchmark interpretation and resource-capacity claims remain qualified as described above. The report is therefore submitted under Track B, which permits AI assistance with disclosure, rather than Track A's no-AI requirement.

References

[1] D. Bakhvalov, *Performance Analysis and Tuning on Modern CPUs*, "Modern CPU Design," Golden Cove TLB hierarchy. https://github.com/dendibakh/perf-book/blob/main/chapters/3-CPU-Microarchitecture/3-8%20Modern%20CPU%20design.md.

[2] Gramine Project, *Gramine Documentation: Running Applications with Intel SGX*. https://gramine.readthedocs.io/en/stable/.

[3] Intel Corporation, "Intel Xeon Gold 5420+ Processor Specifications." https://www.intel.com/content/www/us/en/products/sku/232381/intel-xeon-gold-5420-processor-52-5m-cache-2-00-ghz/specifications.html (accessed 2026-10-30).

[4] A. Frumusanu, "Golden Cove Microarchitecture (P-Core) Examined," *AnandTech*, 2021. https://www.anandtech.com/Show/Index/16881?all=False&cPage=13&page=3&slug=a-deep-dive-into-intels-alder-lake-microarchitectures&sort=0.

[5] C. Lam, "Popping the Hood on Golden Cove," *Chips and Cheese*, 2021. https://chipsandcheese.com/p/popping-the-hood-on-golden-cove.

[6] H. Wong, "Measuring Reorder Buffer Capacity," 2013. https://blog.stuffedcow.net/2013/05/measuring-rob-capacity/.

[7] Gramine Project, "Manifest Syntax: Command-line Arguments and Debug Enclave Settings." https://gramine.readthedocs.io/en/stable/manifest-syntax.html.